

Выполнение внешнего курса stepik 'Введение в Linux'.

Раздел №3

Архитектура компьютеров и операционные системы. Раздел:
Операционные системы

Зевакина Екатерина Романовна

16 мая 2026

Российский университет дружбы народов, Москва, Россия

Раздел 3.1

Задание 1 в 3.1

[рис. @fig-001]

Какую клавишу(и) нужно нажать на клавиатуре, чтобы выйти из редактора vim? Считайте, что вы только что открыли файл и вам сразу понадобилось выйти из редактора.

Выберите один вариант из списка

☒ Абсолютно точно.

- ☐ "q"
- ☐ "Ctrl", затем "x"
- ☐ "Q"
- ☐ "q", затем "Enter"
- ☒ ":", затем "q", затем "Enter"

Рис. 1: Задание 1 в 3.1

Пояснения: Чтобы выйти из редактора vim, необходимо нажать комбинацию ":", затем "q", затем "Enter". Это прописано в справочнике по редактору.

Задание 2 в 3.1

Формулировка задания: [рис. @fig-002]

А для того, чтобы убедиться, что вы разобрались, отметьте ниже **все верные** утверждения про следующую строку:

```
Strange_ TEXT is_here. 2=2 YES!
```

Рис. 2: Задание 2 в 3.1 Формулировка задания

Выполнение задания: [рис. @fig-003]

Выберите все подходящие ответы из списка

☒ Прекрасный ответ.

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ☐ Чтобы попасть в конец строки, нужно одинаковое число нажатий, что на W, что на w
- ☒ В этой строке 9 "слов" (word)
- ☐ Нажимая только на w, нельзя переместить курсор на "."
- ☐ Чтобы попасть в конец строки, нужно совершить больше нажатий на W, чем на w
- ☐ В этой строке 5 "слов" (word)
- ☐ В этой строке 9 "больших слов" (WORD)

Следующий шаг

Решить снова

Пояснения: Чтобы выполнить данное задание, я открыла редактор vim, ввела строку: Strange_ TEXT is_here. 2=2 YES! и на практике проверяла каждое утверждение.

Задание 3 в 3.1

Формулировка задания: [рис. @fig-004]

Предположим, что в текстовом файле записана одна единственная строка:

```
one two three four five
```

и вам нужно преобразовать её в строку

```
three four four four five
```

Какие(ой) из предложенных ниже наборов нажатий клавиш выполнят такое редактирование? В этих наборах нажатие на клавишу Esc обозначается как <Esc> (т.е. знаки "<" и ">" не несут отдельного смысла).

Рис. 4: Задание 3 в 3.1 Формулировка задания

Выполнение задания: [рис. @fig-005]

Выберите все подходящие ответы из списка

☒ Отлично!

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

☐ d2wwywpp

☒ d2w\$blfour four <Esc>

☒ d2wwifour four <Esc>

Пояснения: Чтобы выполнить данное задание, я открыла редактор vim, ввела 6 строк: one two three four five и на практике проверяла каждое утверждение.

Задание 4 в 3.1

[рис. @fig-006]

Предположим, что вы открыли файл в редакторе vim и хотите заменить в этом файле все строки, содержащие слово `Windows`, на такие же строки, но со словом `Linux`. Если в какой-то строке слово `Windows` встречается больше, чем один раз, то заменить на `Linux` в этой строке нужно **только самое первое** из этих слов.

Какую команду нужно ввести для этого в vim? Укажите необходимую команду целиком (т.е. **включая** ввод ":" в самом начале), однако нажатие на `Enter` после ввода команды обозначать никак **не нужно**.

Напишите текст

✓ Верно. Так держать!

`:%s/Windows/Linux`

Следующий шаг

Решить снова

Рис. 6: Задание 4 в 3.1

Пояснения: `:%s/Windows/Linux` находит каждую строку, содержащую «Windows» и заменяет в ней только первое вхождение «Windows» на «Linux»

Задание 5 в 3.1

Формулировка задания: [рис. @fig-007]

Мы совсем не рассказали вам про третий режим работы vim – режим **выделения (Visual)**. Предлагаем вам ознакомиться с ним самостоятельно. Например, это можно сделать во время прохождения упражнений в vimtutor, который мы настоятельно рекомендуем вам для изучения vim!

Чтобы убедиться, что вы разобрались с этим режимом работы, отметьте, пожалуйста, **все верные** утверждения из списка ниже.

Рис. 7: Задание 5 в 3.1 Формулировка задания

Выполнение задания: [рис. @fig-008]

Выберите все подходящие ответы из списка

☒ Отличное решение!

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ☐ Чтобы выйти из режима выделения, нужно ввести :q
- ☒ В режиме выделения можно использовать команды перемещения (например, W, e, \$, и др.)
- ☒ В режиме выделения можно использовать команды d (удалить) и y (скопировать)
- ☒ Режим выделения открывается из нормального режима по нажатию "v"
- ☒ Когда вы находитесь в режиме выделения, внизу редактора горит надпись – VISUAL – (или – ВИЗУАЛЬНЫЙ РЕЖИМ –)
- ☒ Выйти из режима выделения можно, нажав клавишу Esc два раза

Пояснения: Чтобы выполнить данное задание, я открыла редактор vim, вошла в режим visual и на практике проверяла каждое утверждение.

Раздел 3.2

Задание 1 в 3.2

[рис. @fig-009]

Предположим, что вы открыли терминал и у вас в нем запущена оболочка `bash`. Вы набираете в ней команды `A1`, `A2`, `A3`, а затем запускаете оболочку `sh`. В этой оболочке вы набираете команды `B1`, `B2`, `B3` и запускаете оболочку `bash`. И, наконец, в этой последней оболочке вы набираете команды `C1`, `C2`, `C3`. Если теперь вы попытаете при помощи стрелочек вверх/вниз перемещаться по истории набранных команд, то команды из какого набора(ов) будут появляться?

Выберите один вариант из списка

☒ Прекрасный ответ.

- ☒ Только из набора C
- ☐ Только из набора A
- ☐ Никакие команды появляться не будут
- ☐ Из наборов B и C
- ☐ Из наборов A и C

Следующий шаг

Решить снова

Рис. 9: Задание 1 в 3.2

Пояснения: Каждая оболочка имеет собственную изолированную историю команд, хранящуюся в памяти процесса.

Задание 2 в 3.2

Формулировка задания: [рис. @fig-010]

Предположим, что вы находитесь в директории `/home/bi/Documents/` и запускаете в ней скрипт следующего содержания:

```
#!/bin/bash  
  
cd /home/bi/  
touch file1.txt  
cd /home/bi/Desktop/
```

Как будет выглядеть **абсолютный путь** до созданного файла `file1.txt` по окончании работы скрипта?

Рис. 10: Задание 2 в 3.2 Формулировка задания

Выполнение задания: [рис. @fig-011]

Выберите один вариант из списка

☒ Всё получилось!

- ☐ Никак (файла `file1.txt` не будет существовать после завершения работы скрипта)
- ☐ `/home/bi/Desktop/file1.txt`
- ☒ `/home/bi/file1.txt`
- ☐ `/home/bi/Documents/file1.txt`

Пояснения: `cd /home/bi/` - переходим в папку `/home/bi/`; `touch file1.txt` - создаем файл `file1.txt`; `cd /home/bi/Desktop/` - переходим в папку `/home/bi/Desktop/`, но не перемещаем `file1.txt` в эту папку. За это отвечает команда `mv`.

Задание 3 в 3.2

Формулировка задания: [рис. @fig-012]

Какие из представленных ниже строк **могут** быть именами переменных в bash? Выберите **все** подходящие варианты!

Рис. 12: Задание 3 в 3.2 Формулировка задания

Выполнение задания: [рис. @fig-013]

Выберите все подходящие ответы из списка

☒ Верно. Так держать!

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ☒ __variable
- ☒ variable123
- ☐ var-l-able
- ☐ vari/able
- ☒ variable_123
- ☐ var@i-able
- ☒ variable

Следующий шаг

Решить снова

Пояснения: Я следовала правилам, что имя переменной может содержать - цифры, буквы, _ и начинаться с - буквы или _

Задание 4 в 3.2

Формулировка задания: [рис. @fig-014]

Напишите скрипт на bash, который принимает на вход два аргумента и выводит на экран строку следующего вида:

```
Arguments are: $1=первый_аргумент $2=второй_аргумент
```

Например, если ваш скрипт называется `./script.sh`, то при запуске его `./script.sh one two` на экране должно появиться:

```
Arguments are: $1=one $2=two
```

а при запуске `./script.sh three four` будет:

```
Arguments are: $1=three $2=four
```

Рис. 14: Задание 4 в 3.2 Формулировка задания

Выполнение задания: [рис. @fig-015]

Напишите программу. Тестируется через stdin → stdout

✓ Правильно.

Теперь вам доступен [Форум решений](#), где вы можете сравнить свое решение с другими или спросить совета.

Пояснения: `echo "Arguments are: $1=$1 $2=$2"` выводит строку `Arguments are: $1=первый_аргумент $2=второй_аргумент`, где экранирование `$1` и `$2` предотвращает подстановку позиционных параметров в самой команде `echo`, поэтому значения `$1` и `$2` будут подставлены из аргументов, переданных скрипту (или текущей оболочке), а не самой команде `echo`.

Раздел 3.3

Задание 1 в 3.3

Формулировка задания: [рис. @fig-016]

Предположим, вы пишете скрипт на `bash` и хотите использовать в нем конструкцию `if` в следующем фрагменте:

```
if [[ ... ]]
then
  echo "True"
fi
```

Вы можете вписать вместо `"..."` (внутри `[[ ]]` и **не забудьте про пробелы** после `[[` и перед `]]`!) любое из перечисленных ниже условий. Однако мы просим вас выбрать только те из них, при которых `echo` напечатает на экран `True` вне зависимости от того, с какими параметрами был запущен ваш скрипт и какие в нем есть переменные.

Рис. 16: Задание 1 в 3.3 Формулировка задания

Выполнение задания: [рис. @fig-017]

Выберите все подходящие ответы из списка

☒ Так точно!

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

☒ `$var1 == $var2 || $var1 != $var2`

Пояснения: `var1 == var2 && var1 != var2` - всегда ложь, т.к. условия друг друга исключают, `# -gt 0` - не всегда истинно, потому что скрипт или функция могут быть вызваны без аргументов, `n 1` - ложно, если не передан аргумент, `z` " " - всегда ложь, т.к. строка не пуста.

Задание 2 в 3.3

Формулировка задания: [рис. @fig-018]

Посмотрите на фрагмент bash-скрипта:

```
if [[ $var -gt 5 ]]
then
  echo "one"
elif [[ $var -lt 3 ]]
then
  echo "two"
elif [[ $var -eq 4 ]]
then
  echo "three"
else
  echo "four"
fi
```

Какие строки и в какой последовательности он выведет на экран, если сначала этот скрипт запустили задав переменную **var=3**, а затем запустили еще раз, но уже с **var=5**.

Рис. 18: Задание 2 в 3.3 Формулировка задания

Выполнение задания: [рис. @fig-019]

Выберите один вариант из списка

 Отлично!

Пояснения: Эту задачу я решила ручной трассировкой.

Задание 3 в 3.3

Формулировка задания: [рис. @fig-020]

Напишите скрипт на bash, который принимает на вход один аргумент (целое число от 0 до бесконечности), который будет обозначать число студентов в аудитории. В зависимости от значения числа нужно вывести разные сообщения.

Соответствие входа и выхода должно быть таким:

```
0 --> No students
1 --> 1 student
2 --> 2 students
3 --> 3 students
4 --> 4 students
5 и больше --> A lot of students
```

Примечание а): выводить нужно только строку справа, т.е. "-->" выводить не нужно.

Примечание б): в последней строке слово "lot" с маленькой буквы!

Рис. 20: Задание 3 в 3.3 Формулировка задания

Выполнение задания: [рис. @fig-021]

Напишите программу. Тестируется через stdin → stdout

✓ Хорошие новости, верно!

Теперь вам доступен [Форум решений](#), где вы можете сравнить свое решение с другими или спросить совета.

Пояснения: Я написала скрипт на bash, который принимает на вход один аргумент (целое число от 0 до бесконечности), который будет обозначать число студентов в аудитории. В зависимости от значения числа нужно вывести разные сообщения.

Задание 4 в 3.3

Формулировка задания: [рис. @fig-022]

Посмотрите на фрагмент bash-скрипта:

```
for str in a , b , c_d
do
  echo "start"
  if [[ $str > "c" ]]
  then
    continue
  fi
  echo "finish"
done
```

Если запустить этот скрипт, то **сколько раз** на экран будет выведено слово **"start"**, а сколько раз слово **"finish"**?

Рис. 22: Задание 4 в 3.3 Формулировка задания

Выполнение задания: [рис. @fig-023]

Выберите один вариант из списка

☒ Так точно!

☐ 5 раз "start" и 4 раза "finish"

Пояснения: Эту задачу я решила ручной трассировкой.

Задание 5 в 3.3

Формулировка задания: [рис. @fig-024]

Напишите скрипт на `bash`, который будет определять в какую возрастную группу попадают пользователи. При запуске скрипт должен вывести сообщение **"enter your name:"** и ждать от пользователя ввода имени (используйте `read`, чтобы прочитать его). Когда имя введено, то скрипт должен написать **"enter your age:"** и ждать ввода возраста (опять нужен `read`). Когда возраст введен, скрипт пишет на экран **"<Имя>, your group is <группа>"**, где **<группа>** определяется на основе возраста по следующим правилам:

- младше либо равно 16: **"child"**,
- от 17 до 25 (включительно): **"youth"**,
- старше 25: **"adult"**.

После этого скрипт опять выводит сообщение **"enter your name:"** и всё начинается по новой (бесконечный цикл!). Если в какой-то момент работы скрипта будет введено **пустое имя** или **возраст 0**, то скрипт должен написать на экран **"bye"** и закончить свою работу (выход из цикла!).

Рис. 24: Задание 5 в 3.3 Формулировка задания

Выполнение задания: [рис. @fig-025]

Напишите программу. Тестируется через `stdin → stdout`

✓ Хорошие новости, верно!

Теперь вам доступен [Форум решений](#), где вы можете сравнить свое решение с другими или спросить совета.

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё

Пояснения: Я написала скрипт на bash, который будет определять в какую возрастную группу попадают пользователи. При запуске скрипт выводит сообщение “enter your name:” и ждёт от пользователя ввода имени (используйте read, чтобы прочесть его). Когда имя введено, то скрипт пишет “enter your age:” и ждёт ввода возраста (опять нужен read). Когда возраст введен, скрипт пишет на экран “, your group is ”, где определяется на основе возраста по следующим правилам.

Раздел 3.4

Задание 1 в 3.4

Формулировка задания: [рис. @fig-026]

Какие(ая) из предложенных ниже инструкций увеличат значение переменной `a` на значение переменной `b`? Например, если в `a` было записано 10, в `b` было 5, то в `a` должно записаться 15.
Выберите **все подходящие** варианты!

Рис. 26: Задание 1 в 3.4 Формулировка задания

Выполнение задания: [рис. @fig-027]

Выберите все подходящие ответы из списка

☒ Правильно, молодец!

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ☒ let "a+=b"
- ☒ let "a=\$a+\$b"
- ☒ let a=a+b
- ☐ let "a+=b"
- ☐ a+=b

Следующий шаг

Решить снова

Пояснения: let “a+=b” не увеличит a на b, а просто присвоит a значение b; a+=\$d оператор += для целочисленных переменных работает как конкатенация строк, а не как сложение.

Задание 2 в 3.4

Формулировка задания: [рис. @fig-028]

Пусть вы находитесь в директории `/home/bi/Documents/` и запускаете в ней скрипт следующего содержания:

```
#!/bin/bash  
  
cd /home/bi/  
echo "`pwd`"
```

Что в этом случае выведет команда `echo` на экран?

Рис. 28: Задание 2 в 3.4 Формулировка задания

Выполнение задания: [рис. @fig-029]

Выберите один вариант из списка

☒ Прекрасный ответ.

- ☐ ``pwd``
- ☐ `/home/bi/Documents`
- ☐ `pwd`
- ☒ `/home/bi`
- ☐ Код возврата команды `pwd` (0 в случае успешного выполнения и не 0 в случае ошибок)

Пояснения: Строка `pwd` (в обратных кавычках) выполняет подстановку команды: сначала запускается команда `pwd`.

Задание 3 в 3.4

Формулировка задания: [рис. @fig-030]

Однако как быть, если хочется всё-таки запустить программу `program`, которая пишет что-то в `stdout` и потом выполнить какие-то действия если ее код возврата равен 0? Выберите **все верные** утверждения или правильно работающие конструкции `if`.

Примечание: во всех вариантах ответов, где есть кавычка, **используется** именно **косая кавычка** (`'`), а не обычная (`"`) или двойная (`\"`).

Рис. 30: Задание 3 в 3.4 Формулировка задания

Выполнение задания: [рис. @fig-031]

Выберите все подходящие ответы из списка

☒ Всё получилось!

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ☐ Ничего сделать нельзя
- ☐ Сначала `var="program"`, затем `if [[ $var -eq 0 ]]`
- ☐ `if [[ `program` -eq 0 ]]`
- ☒ Сначала запустить `program`, затем `if [[ $? -eq 0 ]]`
- ☒ `if `program` > some_file.txt`

Пояснения: `if [[ -eq 0 ]]`— подстановка вывода программы, и
если `program` выводит не число `0`, условие ложно или выдает ошибку; `if >
some_file.txt- , >(0)`
`program,` `if` с подстановкой.

Задание 4 в 3.4

Формулировка задания: [рис. @fig-032]

Посмотрите на функцию из bash-скрипта:

```
counter () # takes one argument
{
    local let "c1+=${1}"
    let "c2+=${1}*2"
}
```

Впишите в форму ниже **строку**, которую выведет на экран команда `echo "counters are $c1 and $c2"` если она находится в скрипте **после десяти вызовов** функции `counter` с параметрами сначала 1, затем 2, затем 3 и т.д., последний вызов с параметром 10.

Рис. 32: Задание 4 в 3.4 Формулировка задания

Выполнение задания: [рис. @fig-033]

Напишите текст



Абсолютно точно.

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

Пояснения: Функция counter увеличивает глобальную (или локальную, если они объявлены выше) переменную c1 на значение переданного аргумента \$1, а переменную c2 — на удвоенное значение этого аргумента

Задание 5 в 3.4

Формулировка задания: [рис. @fig-034]

Напишите скрипт на bash, который будет искать наибольший общий делитель ([НОД](#), greatest common divisor, GCD) двух чисел. При запуске ваш скрипт не должен ничего писать на экран, а просто ждет ввода двух натуральных чисел через пробел (для этого можно использовать `read` и указать ему две переменные – см. пример в видеофрагменте). После ввода чисел скрипт считает их НОД и выводит на экран сообщение "GCD is <исчисленное значение>", например, для чисел 15 и 25 это будет "GCD is 5". После этого скрипт опять входит в режим ожидания двух натуральных чисел. Если в какой-то момент работы пользователь ввел вместо этого пустую строку, то нужно написать на экран "bye" и закончить свою работу.

Рис. 34: Задание 5 в 3.4 Формулировка задания

Выполнение задания: [рис. @fig-035]

Напишите программу. Тестируется через stdin → stdout

✓ Прекрасный ответ.

Теперь вам доступен [Форум решений](#), где вы можете сравнить свое решение с другими или спросить совета.

```
1 while [ true ]
2 do
3     read n1 n2
4     if [ -z $n1 ]; then
5         echo "bye"
6         break
7     else
```

Пояснения: Я написала скрипт на `bash`, который будет искать наибольший общий делитель двух чисел. При запуске скрипт ничего не пишет на экран, а просто ждет ввода двух натуральных чисел через пробел. После ввода чисел скрипт считает их НОД и выводит на экран сообщение "GCD is ".

Задание 6 в 3.4

Формулировка задания: [рис. @fig-036]

Напишите **калькулятор** на bash. При запуске ваш скрипт должен ожидать ввода пользователем команды (при этом на экран выводить ничего не нужно). Команды могут быть трех типов:

Рис. 36: Задание 6 в 3.4 Формулировка задания

Выполнение задания: [рис. @fig-037]

Напишите программу. Тестируется через stdin → stdout

✓ Так точно!

Теперь вам доступен [Форум решений](#), где вы можете сравнить свое решение с другими или спросить совета.

```
1 #!/bin/bash
2 while [[ True ]]
3 do
4     read birinchi amal ikkinchi
5     if [[ $birinchi == "exit" ]]
6     then
7         echo "bye"
8         break
9     elif [[ "$birinchi" =~ "^[0-9]+$" && "$ikkinchi" =~ "^[0-9]+$" ]]
10    then
11        echo "error"
```

Пояснения: Я написала программу калькулятор на bash.

Раздел 3.5

Задание 1 в 3.5

[рис. @fig-038]

Пусть в директории `/home/bi` лежат файлы `Star_Wars.avi`, `star_trek_OST.mp3`, `STARS.txt`, `stardust.mpeg`, `Eddard_Stark_biography.txt`.

Отметьте все файлы, которые **найдет** команда `find /home/bi -iname "star*"`, но **НЕ найдет** команда `find /home/bi -name "star*"` ?

Выберите все подходящие ответы из списка

☒ Правильно.

- ☐ `star_trek_OST.mp3`
- ☐ `Eddard_Stark_biography.txt`
- ☐ `stardust.mpeg`
- ☒ `Star_Wars.avi`
- ☒ `STARS.txt`

Рис. 38: Задание 1 в 3.5

Пояснения: `star_trek_OST.mp3` - будет найден “star”;

Задание 2 в 3.5

[рис. @fig-039]

Задание на понимание работы опций `-path` и `-name` команды `find`. Отметьте **все верные** утверждения из перечисленных ниже.

Выберите все подходящие ответы из списка

✔ Отличное решение!

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ☐ Опция `-path` используется только для поиска директорий, а `-name` только для поиска файлов
- ☒ В некоторых случаях `find` с `-name` найдет меньше файлов, чем `find` с таким же запросом, но с `-path`
- ☒ В некоторых случаях `find` с `-name` найдет больше файлов, чем `find` с таким же запросом, но с `-path`
- ☐ Если заменить в команде поиска `-name`, на `-path`, то результат поиска всегда останется неизменным
- ☒ Если заменить в команде поиска `-name`, на `-path`, то результат поиска иногда может остаться таким же

Следующий шаг

Решить снова

Рис. 39: Задание 2 в 3.5

Пояснения: Опция `-path` используется только для поиска директорий, а `-name` только для поиска файлов. - неверно, `-path` используется для сопоставления полного пути (включая имя файла или директории), а не только директорий; Если заменить в команде поиска `-name`, на `-path`, то результат поиска всегда останется неизменным - неверно, `-name` применяется только к последнему компоненту имени (имени файла или директории), а `-path` — ко всему пути, поэтому замена `-name` на `-path` изменит критерий поиска и почти всегда даст другой результат.

Задание 3 в 3.5

Формулировка задания: [рис. @fig-040]

Предположим, что в директории `/home/bi/` есть следующая структура файлов и поддиректорий:

```
/home/bi/  
└─ dir1  
   └─ file1  
      └─ dir2  
         └─ file2  
            └─ dir3  
               └─ file3
```

Какие(ой) из трех файлов (`file1`, `file2`, `file3`) будут найдены по команде `find /home/bi -mindepth 2 -maxdepth 3 -name "file*"` ?

Рис. 40: Задание 3 в 3.5 Формулировка задания

Выполнение задания: [рис. @fig-041]

Выберите один вариант из списка

☒ Здорово, всё верно.

☐ Все кроме file2

☐ Все три файла

Пояснения: по команде `find /home/bi -mindepth 2 -maxdepth 3 -name "file*"` будет найден file3.

Задание 4 в 3.5

Формулировка задания: [рис. @fig-042]

Задание на понимание работы опций `-A`, `-B` и `-C` команды `grep`. Пусть у вас есть файл `file.txt` из 10 строк, причем **в каждой строке есть слово "word"**. Если вы выполните на этом файле команды:

```
grep "word" file.txt > results.txt
grep -A 1 "word" file.txt > results.txt
grep -B 1 "word" file.txt > results.txt
grep -C 1 "word" file.txt > results.txt
```

то какая(ие) из них создаст файл `results.txt` наибольшего размера?

Рис. 42: Задание 4 в 3.5 Формулировка задания

Выполнение задания: [рис. @fig-043]

Выберите один вариант из списка

☒ Абсолютно точно.

- ☒ results.txt будет одинакового размера во всех случаях
- ☐ Все, кроме `grep "word" file.txt > results.txt`
- ☐ `grep -C 1 "word" file.txt > results.txt`
- ☐ `grep -A 1 "word" file.txt > results.txt`
- ☐ `grep -A 1 "word" file.txt > results.txt` и `grep -B 1 "word" file.txt > results.txt`

Пояснения: results.txt будет одинакового размера во всех четырёх случаях, только если в файле file.txt каждая строка, содержащая word, не имеет соседних строк (ни сверху, ни снизу), либо если файл состоит ровно из одной строки с word и эта строка является одновременно первой, последней и единственной.

Задание 5 в 3.5

[рис. @fig-044]

Предположим, что в файле `text.txt` записаны строки, показанные среди вариантов ответа. Отметьте только те из них, которые выведет на экран команда `grep -E "[xkLXKL]?[uU]buntu$" text.txt`.

Выберите все подходящие ответы из списка

☒ Здорово, всё верно.

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ☒ The best OS is Xubuntu
- ☒ Hmm, XKLubuntu
- ☒ Mac OS X, Windows, Ubuntu
- ☒ Lubuntu is better than Ubuntu
- ☒ Linux is not always Ubuntu
- ☒ I prefer Kubuntu

Следующий шаг

Решить снова

Рис. 44: Задание 5 в 3.5

Пояснения: Все вышеперечисленные строки подходят под маску

Задание 6 в 3.5

[рис. @fig-045]

Что произойдет, если в команде `sed -n "/[a-z]*/p" text.txt` не указывать опцию `-n` ?

Выберите один вариант из списка

☒ Прекрасный ответ.

- ☐ Появится сообщение об ошибке
- ☒ Каждая строка будет выведена два раза
- ☐ На экран ничего не напечатается
- ☐ Будут выведены все строки файла text.txt, в которых есть только большие буквы латинского алфавита

[Следующий шаг](#) [Решить снова](#)

Рис. 45: Задание 6 в 3.5

Пояснения: Каждая строка будет выведена два раза, если в команде `sed -n "/[a-z]*/p" text.txt` не указывать опцию `-n`.

Задание 7 в 3.5

Формулировка задания: [рис. @fig-046]

Запишите в форму ниже инструкцию `sed`, которая заменит все "аббревиатуры" в файле `input.txt` на слово "abbreviation" и запишет результат в файл `edited.txt` (на экран при этом ничего выводить не нужно). Обратите внимание, что **в инструкции должны быть** указаны и сам `sed`, и оба файла!

Рис. 46: Задание 7 в 3.5 Формулировка задания

Выполнение задания: [рис. @fig-047]

Напишите текст

✓ Верно.

```
sed 's/[A-Z]\{2,\} /abbreviation /g' input.txt > edited.txt
```

Следующий шаг

Решить снова

Рис. 47: Задание 7 в 3.5 Выполнение задания

Пояснения: `sed 's/[A-Z]{2,} /abbreviation /g' input.txt > edited.txt` заменит все “аббревиатуры” в файле `input.txt` на слово “abbreviation” и запишет результат в файл `edited.txt`.

Раздел 3.6

Задание 1 в 3.6

[рис. @fig-048]

Какую опцию нужно указать при запуске `gnuplot`, чтобы при его закрытии не были автоматически закрыты и все нарисованные в нём графики?

Выберите один вариант из списка

☒ Правильно, молодец!

- ☐ -s, --show-plots-after-exit
- ☒ -p, --persist
- ☐ Графики и так не закрываются автоматически при закрытии `gnuplot`!
- ☐ Такой опции не существует

[Следующий шаг](#) [Решить снова](#)

Рис. 48: Задание 1 в 3.6

Пояснения: Флаг `-p` или `--persist` означает «сохранить изменения после перезагрузки» или «не завершать процесс после отключения».

Задание 2 в 3.6

Формулировка задания: [рис. @fig-049]

Предположим у вас есть файл `data.csv` с двумя столбцами по 10 чисел в каждом. В первой строке не записаны названия столбцов, т.е. ряды данных начинаются прямо с первой строки. Вы запускаете `gnuplot` и вводите в него две команды:

```
set key autotitle columnhead
plot 'data.csv' using 1:2
```

Какое в этом случае будет **название** у построенного **ряда данных** и **сколько** будет нарисовано **точек** на графике?

Рис. 49: Задание 2 в 3.6 Формулировка задания

Выполнение задания: [рис. @fig-050]

Выберите один вариант из списка

☒ Прекрасный ответ.

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ☐ Название "noname", нарисовано 10 точек
- ☐ Название "data.csv" using 1:2, нарисовано 10 точек
- ☐ Название – первое значение из первого столбца, нарисовано 10 точек
- ☒ Название – первое значение из второго столбца, нарисовано 9 точек (точка из первой строки пропущена)

Пояснения: Количество точек на графике: будет 9 точек, так как первая строка каждого столбца использована под заголовок, а остальные 9 строк (со 2-й по 10-ю) — как числовые данные.

Задание 3 в 3.6

Формулировка задания: [рис. @fig-052]

Предположим, что вы пишете gniplot-скрипт и у вас в нем есть три переменные `x1`, `x2`, `x3`, в которых записаны координаты важных точек по оси OX (по возрастанию). Вы хотите, чтобы на этой оси было только три деления (т.е. три черточки) в этих самых координатах, а подписи этих делений были оформлены в виде **"point <номер точки>, value <значение соответствующей переменной>".**

Например, для `x1=0`, `x2=10`, `x3=20`, это были бы надписи "point 1, value 0" в точке с координатой 0 по горизонтали, "point 2, value 10" в точке с координатой 10 и "point 3, value 20" в точке с координатой 20.

Или, например, `x1=100`, `x2=150`, `x3=250`, это были бы надписи "point 1, value 100" в точке с координатой 100, "point 2, value 150" в точке с координатой 150 и "point 3, value 250" в точке с координатой 250.

Впишите в форму ниже **одну команду** (т.е. одну строку), которую нужно добавить в скрипт, для выполнения этой задачи.

Рис. 51: Задание 3 в 3.6 Формулировка задания

Выполнение задания: [рис. @fig-053]

Напишите текст

✓ Верно. Так держать!

```
set xtics ("point 1, value ".x1 x1, "point 2, value ".x2 x2, "point 3, value ".x3 x3)
```

Пояснения: `set xtics ("point 1, value".x1 x1, "point 2, value".x2 x2, "point 3, value".x3 x3)` задаёт пользовательские метки (подписи) на оси X в позициях с координатами `x1`, `x2`, `x3`.

Задание 4 в 3.6

Формулировка задания: [рис. @fig-054]

Указанные файлы использовались в последнем видеофрагменте для создания вращающегося графика. Измените инструкции в файле `move.rot` (т.е. **добавлять** и **удалять** инструкции **нельзя!**) таким образом, чтобы:

- График **отразился зеркально** относительно горизонтальной поверхности. То есть там, где была точка (10, 10, 200), станет точка (10, 10, -200), где была точка (-10, -10, 200) станет (-10, -10, -200) и т.д. При этом точка (0, 0, 0) останется на месте.
- Изображение стало **вращаться в обратную сторону**. То есть если раньше вращалось "влево", то теперь станет "вправо".
- Вращение стало **в два раза быстрее**. То есть станет в два раза больше перерисовок графика на каждую секунду вращения.

Измененный файл загрузите в форму ниже.

Рис. 53: Задание 4 в 3.6 Формулировка задания

Выполнение задания: [рис. @fig-055]

Напишите текст

✔ Отлично!

```
a=a+1
zrot=(zrot+350)%360
set view xrot,zrot
splot -x**2-y**2
pause 0.1
if (a<50) reread
```

Пояснения: Я изменила инструкции в файле `move.rot`, чтобы они подходили под условия задачи.

Раздел 3.7

Задание 1 в 3.7

[рис. @fig-056]

Какая команда(ы) установят файлу `file.txt` права доступа `rwXrw-r--`, если изначально у него были права `r--r--r--`. Укажите **все верные** варианты ответа!

Примечание: запись вида `команда1; команда2; команда3` означает, что в терминале последовательно выполнились все три команды (сначала `команда1`, затем `команда2` и, наконец, `команда3`).

Выберите все подходящие ответы из списка

☒ Здорово, всё верно.

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ☐ `chmod u-wx file.txt; chmod g-w file.txt`
- ☒ `chmod 764 file.txt`
- ☐ `chmod 467 file.txt`
- ☐ `chmod 777 file.txt`
- ☐ `chmod o-wx file.txt; chmod g-x file.txt; chmod a+wx file.txt`
- ☐ `chmod rwxrw-r-- file.txt`

Следующий шаг

Решить снова

Рис. 55: Задание 1 в 3.7

Пояснения: `chmod u-wx file.txt` - снимает у владельца права на запись и выполнение, у группы право на запись; `chmod 467 file.txt`, `chmod 777 file.txt` - не подходят под условие задачи, `chmod o-wx file.txt`; `chmod g-x file.txt`; `chmod a+wx file.txt` - у других снимаются права на запись и выполнение, у группы снимается право на выполнение, Всем добавляются права на запись и выполнение, `chmod rwxrw-r- file.txt` - неправильный синтаксис.

Задание 2 в 3.7

Формулировка задания: [рис. @fig-057]

Предположим вы использовали команду `sudo` для создания директории `dir`. По умолчанию для `dir` были выставлены права доступа `rwxt-xr-x` (владелец `root`, группа `root`). Таким образом никто кроме пользователя `root` не может ничего записывать в эту директорию, например, не может создавать файлы в ней.

После выполнения какой команды `user` из группы `group` всё-таки сможет создать файл внутри `dir`? Укажите **все верные** варианты ответов!

Рис. 56: Задание 2 в 3.7 Формулировка задания

Выполнение задания: [рис. @fig-058]

Выберите все подходящие ответы из списка

☒ Абсолютно точно.

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ☒ `sudo chown user dir`
- ☒ `sudo chown user:group dir`
- ☐ `sudo chmod g+w dir`
- ☒ `sudo chmod o+w dir`
- ☐ `sudo chmod o+x dir`
- ☒ `sudo chmod a+w dir`

Пояснения: `sudo chmod g+w dir` добавляет группе право на запись в директорию `dir`; `sudo chmod o+x dir` добавляет остальным пользователям право на выполнение для `dir`.

Задание 3 в 3.7

[рис. @fig-059]

Отметьте какие характеристики файла можно посчитать с использованием команды `wc`.

Выберите все подходящие ответы из списка

☒ Здорово, всё верно.

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ☒ Количество слов
- ☒ Количество строк
- ☐ Количество предложений
- ☐ Количество определенных букв (например, количество букв "А")
- ☒ Длину самой длинной строки

[Следующий шаг](#) [Решить снова](#)

Рис. 58: Задание 3 в 3.7

Пояснения: `wc` не может посчитать количество предложений и количество

Задание 4 в 3.7

Формулировка задания: [рис. @fig-060]

Впишите в форму ниже команду, которая выведет сколько места на диске занимает текущая директория (при этом **размер** нужно вывести **в удобном для чтения формате** (например, вместо `2048 байт` надо вывести `2.0K`) и **больше** на экран выводить **ничего не** нужно). В команде указывайте **только необходимые** для выполнения задания **опции и аргументы**, лишних опций указывать не нужно!

Рис. 59: Задание 4 в 3.7 Формулировка задания

Выполнение задания: [рис. @fig-061]

Напишите текст



Абсолютно точно.

```
du -h -s
```

Следующий шаг

Решить снова

Рис. 60: Задание 4 в 3.7 Выполнение задания

Пояснения: Команда `du -h -s` выводит общий размер (-s, summary) указанного файла или директории в «человекочитаемом» формате (-h, human-readable)

Задание 5 в 3.7

[рис. @fig-062]

Впишите в форму ниже максимально короткую команду (т.е. в которой минимально возможное число символов), которая позволит создать в текущей директории 3 поддиректории с именами `dir1`, `dir2`, `dir3`.

Если вы придумали команду, которая выполняет эту задачу, а система проверки сообщает вам "Incorrect"/"Неверно", то скорее всего вы придумали не самую короткую команду из возможных!

Напишите текст

✓ Хорошая работа.

`mkdir dir{1..3}`

Следующий шаг Решить снова

Рис. 61: Задание 5 в 3.7

Пояснения: `mkdir dir{1..3}` - максимально короткая команда, которая позволяет создать в текущей директории 3 поддиректории с именами `dir1`,